

# Symphonym: Universal Phonetic Embeddings for Cross-Script Toponym Matching via Teacher-Student Distillation

Stephen Gadd

ORCID: 0000-0003-3060-0181

School of Advanced Study, University of London / University of Pittsburgh  
stephen.gadd@sas.ac.uk

Draft: January 31, 2026

**Keywords:** toponym matching; phonetic embeddings; cross-script; teacher-student distillation; neural networks; digital humanities; geographic information retrieval; Symphonism

## Current Development Status

This document describes version 6 (v6) of Symphonism, which addresses critical coverage gaps discovered during v5 training. The following summarizes completed improvements and remaining tasks before v6 training can commence.

### Completed: Epitran Extension Program

**Problem Identified.** During v5 training data generation (January 2026), analysis revealed that 8 scripts present in the WHG corpus had zero PanPhon embeddings despite having substantial toponym counts: Armenian (153k toponyms), Greek (218k), Gujarati (21k), Hebrew (152k), Hiragana (152k), Kannada (43k), Katakana (341k), and a heterogeneous “OTHER” category (343k). This resulted in approximately 1.3 million toponyms being excluded from the training process, creating systematic bias in the final model.

**Root Cause.** Epitran<sup>1</sup> (the rule-based grapheme-to-phoneme converter used to generate IPA transcriptions for PanPhon feature extraction) lacked phonetic mapping tables for many language-script pairs represented in WHG. While Epitran officially supports 100+ languages, the specific combinations needed for historical toponymy (e.g., ell-Grek for Modern Greek, hye-Armn for Armenian) were not included.

---

<sup>1</sup><https://github.com/dmort27/epitran>

**Solution Implemented.** Between mid-January and late January 2026, I extended Epitran by creating 97 new language-script mapping CSV files stored in `phonetics/epitran_extensions/`. These mappings cover:

- **Critical missing scripts:** Armenian (`hye-Armn`), Greek (`ell-Grek`), Gujarati (`guj-Gujr`), Georgian (`kat-Geor`), and numerous Indic and non-Latin scripts.
- **High-frequency under-resourced language pairs:** Including Norwegian Bokmål (`nob-Latn`), Scottish Gaelic (`gla-Latn`), Basque (`eus-Latn`), Egyptian Arabic (`arz-Arab`), and 80+ others with toponym counts ranging from 10,000 to 450,000.
- **Specialized historical and minority languages:** Such as Old Prussian (`prg-Latn`), Cajun French (`frc-Latn`), Romansh (`roh-Latn`), and various creoles.

The creation of these 97 custom Epitran extension files was assisted by large language models (Claude and ChatGPT), which generated initial IPA mappings based on linguistic descriptions and Unicode character tables. None of these outputs has been validated by human experts; they serve as provisional mappings to enable v6 training. Future work may involve expert review and refinement of these rules.

The `TODO.txt` file tracks the status of 2,339 unique language-script pairs found in WHG, categorized as:

- EPITRAN: Natively supported by upstream Epitran (34 pairs)
- EPITRAN-EXT: Custom mappings created in this extension program (97 pairs)
- CHARSIU: Chinese topolects (Gan, Wu, Yue) to be handled by CharsiuG2P fallback
- SYMPHONY: Low-frequency pairs (<10k toponyms) where the model will learn from related scripts
- UNHANDLED: Pairs requiring language detection or further investigation
- PHONIKUD?: Hebrew (145k toponyms)—unpointed Hebrew text lacks vowels, making deterministic IPA transcription impossible without a vocalization system like Nikud or a neural fallback

**Note on Chinese Topolects and Hebrew.** Three Chinese varieties (Wu, Gan, Yue) with 49k, 37k, and 33k toponyms respectively cannot be reliably mapped to IPA using rule-based approaches due to insufficient linguistic resources. These

will be handled via CharsiuG2P (Zhu et al., 2022), a neural G2P model. Similarly, Hebrew toponyms present a unique challenge: unpointed Hebrew script (the standard in historical documents) omits vowels, making rule-based phonetic transcription unreliable. For v6, Hebrew text will be processed using Phonikud (Kolani et al., 2025)<sup>2</sup>, which directly generates International Phonetic Alphabet (IPA) transcriptions from unpointed Hebrew text using a neural model. The resulting IPA strings are then passed to PanPhon for generation of 192-dimensional articulatory feature vectors. This two-stage pipeline (Phonikud → PanPhon) enables accurate phonetic embedding generation for the 145k Hebrew toponyms without requiring intermediate Epitran conversion.

## Remaining Work for v6

**Vocabulary Recoding (Completed).** The character vocabulary has been re-generated for v6 to support native script processing. Previously (v4/v5), Chinese, Japanese, and Korean scripts were handled via AnyAscii romanization. With Epitran extensions now providing direct support (cmn-Hans for Mandarin, jpn-Hira/jpn-Kana for Japanese, kor-Hang for Korean Hangul), the vocabulary generation step in `phonetics/extraction/rebuild_toponyms_index.py` has been updated to:

- Remove romanization logic for CJK, Hiragana, Katakana, and Hangul (previously defined in `ROMANIZE_SCRIPTS` in `phonetics/utils/script_detection.py`)
- Expand Unicode range coverage to include full character sets for newly-supported scripts: Armenian (U+0530–U+058F), Greek Extended (U+1F00–U+1FFF), Gujarati (U+0A80–U+0AFF), Kannada (U+0C80–U+0CFF)
- For CJK scripts, include Han characters from the CJK Unified Ideographs block (U+4E00–U+9FFF), but *only those actually observed* in the 67M toponym corpus. While this Unicode range contains approximately 20,900 characters, the WHG corpus uses fewer than 3,000 unique Han characters, as most toponyms utilize a small subset of common logographs. This selective inclusion avoids computational overhead from rare archaic characters while ensuring 100% coverage of observed names.
- Similarly for Korean, include only the observed Hangul syllables and Jamo components rather than all 11,172 possible syllable blocks (U+AC00–U+D7AF)
- Regenerate `char_vocab.json` to include native characters from Armenian, Greek, Gujarati, Korean, Japanese, and other extended scripts instead of their romanized forms
- Update `EPITRAN_SUPPORTED_SCRIPTS` to accurately reflect the 97 new language-script mappings plus native CJK/Hangul/Kana support

---

<sup>2</sup><https://github.com/thewhiteagle/phonikud>

The updated vocabulary size has increased from approximately 11,122 characters (v4/v5) to an estimated 13,500–15,000 characters. This modest increase—despite adding thousands of potential CJK characters—reflects two factors: (1) the shift from romanized to native script representation replaces rather than supplements existing characters, and (2) inclusion of only corpus-observed characters rather than full Unicode blocks. The vocabulary comprises every unique character actually appearing in the training data, ensuring zero out-of-vocabulary (OOV) errors while avoiding memory waste on unused symbols. Character embeddings remain efficient due to the BiLSTM’s ability to learn compositional representations from the observed character set.

**Data Pipeline Verification.** The Elasticsearch toponyms schema has been updated to store Symphonism embeddings using byte quantization (`element_type: byte, int8`) instead of `float32`. This 75% memory reduction (from 512 bytes to 128 bytes per 128-dimensional embedding) is critical for WHG’s production deployment, where the full toponyms index is held in RAM to ensure sub-second query latency for interactive reconciliation operations. The quantization preserves relative distances between embeddings—the essential property for cosine similarity-based KNN retrieval—with negligible impact on ranking quality. Before initiating v6 training, the following verification steps remain:

1. Re-run `rebuild_toponyms_index.py` with updated Epitran extensions, Phonikud, and CharsiuG2P to generate PanPhon embeddings for previously-excluded toponyms
2. Verify via `testing/inspect_pairs.py` that all 20 core scripts now have non-zero training pair counts
3. Re-run `generate_training_data.py` to create v6 training triplets incorporating the expanded coverage
4. Update training statistics in Section ?? to reflect the increased script diversity and pair counts

**Expected Impact.** Full Epitran extension coverage is projected to increase the usable toponym count from 29M (v5) to approximately 31-32M, adding critical representation for Greek, Hebrew, Armenian, and Gujarati toponyms. This should eliminate the script-specific performance gaps observed in v5 MEHDIE testing (Section ??), where Greek and Hebrew pairs showed anomalously low similarity scores due to insufficient training data.

**Note:** This section documents the development process and will be removed before final publication. The implementation details are recorded here to maintain an accurate technical audit trail during the iterative development phase.

## Abstract

Linking place names across historical sources, languages, and writing systems remains a fundamental challenge in digital humanities and geographic information retrieval. Existing approaches either require language-specific phonetic algorithms, expert-curated transliteration rules, or fail to capture the phonetic relationships that make “Moscow”, “Москва”, and “موسكو” recognisably the same place. The objective of this research is to present **Symphonym**, a neural embedding system that maps toponyms from any script into a unified 128-dimensional phonetic space, enabling direct similarity comparison without runtime phonetic conversion or language-specific resources. Symphonism uses a Teacher-Student architecture: a Teacher network trained on articulatory phonetic features (derived via Epitran grapheme-to-phoneme (G2P) conversion and PanPhon feature extraction) produces target embeddings, while a Student network learns to approximate these embeddings directly from characters. The Student operates at inference time without requiring phonetic resources, enabling deployment at scale. Training uses co-located toponym pairs from GeoNames, Wikidata, and the Getty Thesaurus of Geographic Names (TGN), with phonetic similarity filtering to exclude false cognates. The system is optimised for *cross-script* matching—linking “北京” to “Beijing”—where traditional string metrics fail entirely; same-script cross-language matching (London/Londres) is handled adequately but can be supplemented by conventional edit-distance methods in a hybrid pipeline. The system achieves 93% accuracy on cross-script equivalents and 87.5% Recall@1 on the MEHDIE (Multi-Entity Hebrew-to-Arabic Data Interchange Exchange) Hebrew-Arabic benchmark, outperforming Levenshtein (81.5%) and Jaro-Winkler (78.5%) baselines. Symphonism effectively addresses the challenge of cross-script toponym matching by providing a phonetically grounded embedding space, demonstrating strong performance on key benchmarks. Its deployment in the World Historical Gazetteer (WHG) will enable phonetic search and reconciliation across 67M+ toponyms, offering a scalable solution for digital humanities research.

## 1 Introduction

Historical gazetteers aggregate place references from sources spanning millennia, dozens of languages, and multiple writing systems. The World Historical Gazetteer<sup>3</sup> (WHG) alone indexes over 112 million toponym records from antiquity to the present, drawn from sources as diverse as classical Latin itineraries, medieval Arabic geographies, and modern national mapping agencies. Each toponym record is tagged with language and script metadata, though the same surface form (e.g., “London”) may appear multiple times when attested in different languages. Linking these references—determining that “Londinium”, “Londres”, “Лондон”, and “伦敦” all denote the same city—is essential for cross-cultural historical research but remains technically challenging.

The core difficulty is that place name similarity is fundamentally *phonetic*, not orthographic. English speakers recognise “Munich” and “München” as the

---

<sup>3</sup><https://whgazetteer.org>

same city not because the spellings match, but because the sounds are similar enough to suggest common origin. Yet computational approaches to toponym matching have largely relied on string-based metrics (edit distance, Jaro-Winkler) or phonetic algorithms designed for single languages (Soundex, Metaphone for English; Cologne phonetic for German). These approaches fail catastrophically when names cross script boundaries: no amount of edit distance tuning will link “東京” to “Tokyo”.

Recent work has applied neural embeddings to geographic entity matching, but existing systems either operate within single scripts, require language identification at inference time, or depend on curated transliteration resources that don’t scale to the long tail of historical languages. Until now, there has been no system that can place “Москва”, “Moscow”, “موسكو”, and “モスクワ” (Japanese Katakana) near each other in embedding space using only the raw character sequences as input.

This paper presents such a system, which I call *Symphonym*. The key contributions are:

1. A **script-aware but script-agnostic embedding architecture** that maps toponyms from 20+ writing systems into a unified 128-dimensional space where proximity reflects phonetic similarity.
2. A **Teacher-Student training framework** that transfers phonetic knowledge from articulatory features (available only for IPA-transcribed names) to a character-level model requiring no phonetic resources at inference time.
3. A **noise-aware training regime** with strategic language dropout that forces the model to learn script-intrinsic phonetic patterns rather than relying on language-specific shortcuts.
4. **Phonetic similarity filtering** for training data that prevents the model from learning false equivalences between unrelated exonyms (e.g., “Germany”  $\neq$  “Deutschland”).
5. A **three-phase curriculum** progressing from phonetic feature learning through knowledge distillation to hard negative discrimination.

The resulting system will enable fuzzy phonetic search across the full WHG corpus without language identification, script-specific preprocessing, or runtime phonetic conversion. Beyond cross-script matching, the phonetic embedding approach naturally handles *historical orthographic variation*—the inconsistent spellings characteristic of pre-standardisation documents—by grouping sound-alike variants near their modern canonical forms.

**Scope and Integration.** This work addresses the *name-matching* component of toponym resolution specifically. The model has no access to geographic coordinates or spatial context—it operates purely on phonetic similarity between strings. Both Symphonism and WHG’s new reconciliation pipeline (termed WHG PLACE: Place Linkage, Alignment, and Concordance Engine) build on architecture developed by [Grossner and Mostern \(2021\)](#). Within the Place Linkage, Alignment, and Concordance Engine (PLACE), phonetic similarity serves as one input to a larger process: candidate toponyms are first retrieved via the phonetic-aware toponym index, then places containing matching toponyms are selected from the places index and filtered by geographic proximity and other contextual criteria. The contribution here is the phonetic similarity component alone.

In practical terms, Symphonism enhances WHG’s Reconciliation Service API<sup>4</sup> (v2.0), facilitating the work of digital humanities and GLAM (galleries, libraries, archives, museums) professionals who seek to link place references in historical documents, archival catalogues, and research datasets to WHG’s consolidated index. Once linked, these references become entry points to a web of related materials—other sources attesting the same places, variant historical names, and cross-references across collections—even when the original materials span multiple scripts, languages, and historical periods.

## 2 Related Work

### 2.1 Classical Phonetic Algorithms

The earliest computational approaches to phonetic matching emerged from English-language record linkage. Soundex ([Russell, 1918](#)), developed for US Census indexing, maps names to four-character codes by retaining the first letter and encoding subsequent consonants. Its English-centricity is explicit: the algorithm assumes Latin script and English phoneme-grapheme correspondences. Metaphone ([Philips, 1990](#)) and Double Metaphone ([Philips, 2000](#)) improve accuracy for English by handling common spelling variations, but remain fundamentally tied to English phonology.

Language-specific variants exist for other European languages, but each requires separate implementation and fails outside its target language family. No classical algorithm handles cross-script matching: they assume a single alphabet and a known phoneme inventory.

### 2.2 String Similarity Metrics

Edit distance metrics (Levenshtein, Damerau-Levenshtein, Jaro-Winkler) measure orthographic rather than phonetic similarity. While useful for detecting OCR errors or spelling variants within a script, they cannot capture phonetic relation-

---

<sup>4</sup><https://docs.whgazetteer.org/content/technical/apis.html#reconciliation-service-api>

ships across scripts. The Levenshtein distance between “Moscow” and “Москва” is undefined without transliteration; even after romanisation to “Moskva”, the distance of 2 fails to reflect the near-identical pronunciation.

Recchia and Louwerse (Recchia and Louwerse, 2013) evaluated 21 similarity measures on romanised toponyms from 11 countries, finding that optimal measures varied by language but were consistent within linguistic families. Santos et al. (Santos et al., 2018) demonstrated that combining multiple string metrics with machine learning classifiers improved matching accuracy for Portuguese toponyms.

Phonetic-aware edit distances (Kondrak, 2000) weight substitutions by articulatory similarity, but still require input in a common alphabet.

## 2.3 Neural Approaches to Entity Matching

Recent work applies neural embeddings to geographic entity resolution. Qiu et al. (Qiu et al., 2024) proposed GeoBERT, combining a BERT model pre-trained on Chinese geographic text with the Enhanced Sequential Inference Model (ESIM) architecture for toponym pair classification. Their approach integrates multiple Chinese-specific features including Pinyin romanisation, Wubi keyboard encoding, character radicals, and stroke counts. On Chinese address matching datasets, GeoBERT achieved F1 scores exceeding 0.97. However, the Chinese-specific features have no analogues in other writing systems, and the geographic pre-training corpus is monolingual, limiting cross-lingual application.

Most relevant is the MEHDIE project (Sagi et al., 2025), which developed tools for matching toponyms between historical Hebrew and Arabic sources. Sagi et al. present a benchmark comprising place pairs from 13th-century Arabic geographical texts and 12th-century Hebrew travel narratives. Their approach investigates direct transliteration between Hebrew and Arabic scripts using Phonetisaurus (Novak et al., 2016) for grapheme-to-phoneme conversion and a custom rule-based transliteration library (Translit) that exploits the phonetic affinity between Semitic languages.

The MEHDIE work demonstrates that direct transliteration between related scripts often outperforms romanisation, which loses phonetic information in standard Latin schemes. However, their approach requires language-pair-specific rules: the Hebrew-Arabic transliteration exploits shared Semitic phonology that does not generalise to unrelated language pairs. In contrast, my use of EpiTran for Grapheme-to-Phoneme (G2P) conversion supports over 100 languages with a unified IPA output, and PanPhon provides language-agnostic articulatory features. This enables my model to learn phonetic relationships across arbitrary script pairs without requiring manual transliteration rules for each combination.

Rama (Rama, 2016) applied Siamese convolutional networks to cognate identification using character embeddings, demonstrating that neural architectures can learn sound correspondences between related languages. The key distinction from my work lies in the *phonetic priors* provided to the model:



- **Rama (Implicit Learning):** The Siamese CNN operates on raw character embeddings, requiring the model to learn from scratch that characters like ‘b’ and ‘p’ are phonetically related by observing them in similar contexts across training examples. This implicit learning demands large amounts of training data and may fail to generalise to low-resource language pairs.
- **Symphonym (Explicit Features):** By using PanPhon articulatory features, I provide the model with vectors that already encode phonetic properties—both ‘b’ and ‘p’ are marked as +bilabial, -continuant, etc. The model need not learn these relationships; it is *informed* of them, enabling better generalisation to scripts and languages with limited training data.

This distinction is particularly important for toponym matching, where the long tail of historical languages and rare scripts cannot be adequately covered by implicit learning alone.

Most directly relevant to my architectural choices is the PWESUITE work of [Zouhar et al. \(2024\)](#), which systematically evaluates phonetic word embedding methods using PanPhon articulatory features. Their triplet margin model—an LSTM encoder trained with triplet loss where positive/negative pairs are determined by articulatory distance—achieves the best overall performance across their evaluation suite, outperforming metric learning, autoencoder, and count-based approaches. Critically, they demonstrate that using articulatory feature vectors as input substantially improves cross-language transfer compared to IPA or character inputs (Table 2 in their work), and that models trained on one language generalise well to others when grounded in articulatory features (their Figure 3). These findings directly inform my Teacher network design. However, PWESUITE addresses monolingual word embeddings within single scripts; it does not tackle the cross-script matching problem central to toponym resolution. My contribution extends their validated approach—triplet loss with articulatory supervision—to the multilingual, multi-script setting, adding the Teacher-Student distillation that enables inference without phonetic resources and the cross-script training regime that learns to align representations across writing systems.

## 2.4 Cross-Lingual Representation Learning

The broader literature on cross-lingual embeddings provides relevant techniques. [Conneau et al. \(Conneau et al., 2018\)](#) show that monolingual embedding spaces can be aligned post-hoc using small bilingual dictionaries. [Artetxe et al. \(Artetxe et al., 2018\)](#) demonstrate unsupervised alignment using only monolingual corpora. However, these approaches target semantic similarity (“dog” near “chien”) rather than phonetic similarity (“Paris” near “Париж”).

The Transliteration resource (TRANSLIT) ([Benites et al., 2020](#)) provides 1.6 million name variants across 180 languages for training transliteration systems. While valuable for capturing conventionalised transliteration standards—the specific ways humans have historically decided to map sounds between scripts, which

don't always follow strict phonetic logic—TRANSLIT focuses primarily on *personal names* rather than toponyms. More significantly, Symphonism's architecture sidesteps much of the transliteration variance that TRANSLIT addresses: Chinese, Japanese, and Korean (CJK) characters are uniformly romanised via AnyAscii (eliminating Wade-Giles vs. Pinyin distinctions), Hangul is decomposed to Jamo, and non-Latin scripts like Cyrillic, Arabic, and Greek are processed natively through PanPhon rather than romanised. This means competing romanisation conventions (e.g., Gorbachev/Gorbachyov/Gorbachev) are handled at the phonetic level, where the underlying sounds are equivalent regardless of Latin spelling choices.

## 2.5 Phonetic Representations in NLP

The use of phonetic features in natural language processing has a long history. Epitran (Mortensen et al., 2018) provides rule-based grapheme-to-phoneme conversion for over 100 languages, outputting IPA transcriptions. PanPhon (Mortensen et al., 2016) maps IPA segments to articulatory feature vectors encoding phonological properties. These resources form the foundation of my Teacher network's phonetic grounding.

## 2.6 Siamese Networks for Similarity Learning

Siamese networks (Bromley et al., 1993; Chopra et al., 2005) learn similarity functions by processing two inputs through identical sub-networks and comparing their output representations. They have been successfully applied to face verification (Schroff et al., 2015), signature verification, and text similarity (Mueller and Thyagarajan, 2016; Neculoiu et al., 2016).

For text matching, Mueller and Thyagarajan (Mueller and Thyagarajan, 2016) demonstrated that Siamese Long Short-Term Memory (LSTM) networks with Manhattan distance ( $L_1$ ) achieve strong performance on sentence similarity tasks. However, the choice of distance metric carries important implications for toponym matching:

**Manhattan Distance Limitations.** The  $L_1$  norm is additive across dimensions, which creates a *length penalty*: toponyms varying significantly in character count (e.g., “Oa” vs. “Constantinople”) may produce large distances purely due to embedding magnitude rather than phonetic dissimilarity. Additionally, Manhattan distance assumes that the optimal path between embeddings follows axis-aligned grid lines, which poorly captures the directional relationships in phonetic space where the *ratio* of articulatory features often matters more than their absolute magnitudes.

**Cosine Similarity Advantages.** Cosine similarity normalises for vector magnitude, measuring only the angle between embeddings. For phonetic representations, this is preferable: two names with similar phonetic “direction” (ratio of

nasality to voicing, place to manner, etc.) should be considered similar regardless of sequence length. I therefore use cosine similarity for inference and incorporate it into the training loss.

Lin et al. (Lin et al., 2017) introduced structured self-attention for sentence embeddings. My architecture extends this line of work by incorporating self-attention mechanisms and operating on phonetic feature sequences, with distance metrics chosen specifically for the phonetic domain.

## 2.7 Summary of Technical Distinctions

Table 1 summarises the key technical distinctions between my approach and related neural matching methods.

| Feature            | Rama (2016)          | Mueller & Thyagarajan    | Sagi et al. (MEHDIE)   | Symphonym                       |
|--------------------|----------------------|--------------------------|------------------------|---------------------------------|
| Input              | Character embeddings | Word/sentence embeddings | Transliterated strings | Articulatory features (PanPhon) |
| Phonetic Knowledge | Implicit (learned)   | None (semantic)          | Rule-based translit.   | Explicit (engineered)           |
| G2P Method         | None                 | None                     | Phonetisaurus          | Epitran (100+ langs)            |
| Distance Metric    | Contrastive (Eucl.)  | Manhattan ( $L_1$ )      | Edit distance          | Cosine + triplet                |
| Scripts            | Single               | Single                   | Hebrew–Arabic          | 20 unified                      |
| Strength           | Minimal preproc.     | Gradient stability       | Semitic affinity       | Cross-lingual                   |

Table 1: Comparison of neural matching approaches. Symphonism differs from prior work in its *learned* (vs. rule-based) phonetic representations and unified multi-script architecture enabling cross-script matching without transliteration.

## 3 Architecture

My system employs a Teacher-Student architecture where a Teacher network, trained on articulatory phonetic features, produces target embeddings that a Student network learns to approximate from character sequences alone. At inference time, only the Student is required, enabling deployment without phonetic resources. This design addresses the limitations of prior approaches discussed in Section 2, particularly the reliance on language-specific phonetic rules and single-script operation.

### 3.1 Design Principles

Three principles guide my architecture:

**Script Awareness Without Script Dependence.** The model must handle 20 writing systems but produce embeddings in a unified space where script boundaries are transparent. I achieve this through deterministic script detection (via Unicode block analysis) combined with learned script embeddings that allow the model to interpret characters appropriately for each script.

**Phonetic Grounding.** Embedding similarity must reflect phonetic similarity, not orthographic or semantic similarity. I ground the embedding space through the Teacher network, which operates on articulatory features derived from IPA transcriptions.

**Inference Efficiency.** The deployed model must encode toponyms without runtime phonetic conversion, language identification, or external resources. All phonetic knowledge is distilled into the Student’s parameters during training.

### 3.2 Architecture Overview

Figure 1 illustrates the Teacher-Student architecture and the three-phase training curriculum.

### 3.3 Script Detection and Character Vocabulary

I detect script from Unicode code points, mapping each character to one of 20 script categories: Latin, Cyrillic, Greek, Arabic, Hebrew, Devanagari, Bengali, Tamil, Telugu, Malayalam, Kannada, Gujarati, Thai, Georgian, Armenian, Chinese (Han), Japanese (Hiragana, Katakana), Korean (Hangul), and Other. Analysis of the 66.9 million toponyms reveals 8,448 unique observed characters across these scripts.

Character vocabulary construction balances coverage against tractability:

- **Alphabetic scripts:** Native characters are retained with expansion to full Unicode ranges for each observed script. This yields script-specific vocabularies ranging from 72 tokens (Tamil) to 1,368 tokens (Arabic extended), with intermediate sizes for Cyrillic (450), Greek (373), Devanagari (162), Georgian (128), and others.
- **Chinese and Japanese:** The tens of thousands of Han characters would create an intractable vocabulary. I romanise via AnyAscii, reducing to the ASCII character set while preserving approximate phonetic content. Hiragana and Katakana are similarly romanised.
- **Korean Hangul:** Rather than treating 11,172 syllable blocks as atomic units, I decompose into Jamo (lead consonant, vowel, optional tail consonant), yielding 51 Jamo tokens with explicit phonetic structure.

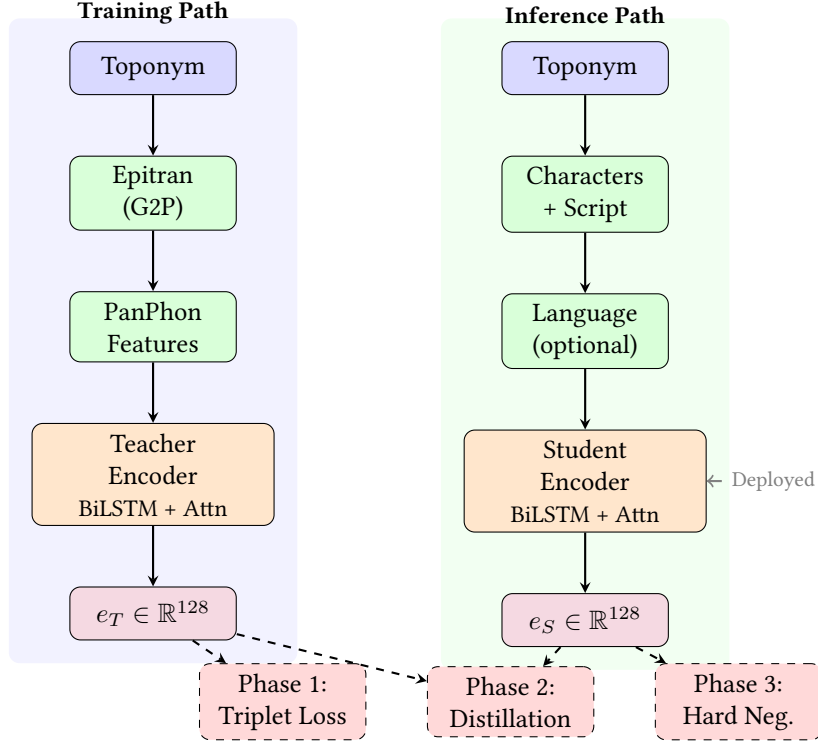


Figure 1: Teacher-Student architecture. **Left:** The Teacher pathway converts toponyms to IPA via Epitrans, extracts PanPhon articulatory features, and encodes to 128-d embeddings. Used only during training. **Right:** The Student pathway processes raw characters with script/language metadata. Deployed at inference time without phonetic resources. **Bottom:** Three training phases progressively transfer phonetic knowledge from Teacher to Student.

The resulting vocabulary contains 11,122 tokens: 94 ASCII characters (for romanised CJK and base Latin), 51 Korean Jamo, and script-specific character sets totalling approximately 10,977 tokens across 17 native scripts. Each token is tagged with its source script for embedding lookup. The vocabulary additionally supports 1,944 distinct language codes observed in the corpus.

### 3.4 Teacher Network: Phonetic Feature Encoder

The Teacher network encodes toponyms via their IPA transcriptions and articulatory features. Given a toponym, I first obtain an IPA transcription using Epitrans (Mortensen et al., 2018), then extract PanPhon features (Mortensen et al., 2016)—a 24-dimensional binary vector per phoneme encoding articulatory properties (place, manner, voicing, etc.).

The Teacher architecture:

1. **Input:** Sequence of PanPhon feature vectors,  $(f_1, f_2, \dots, f_T)$  where  $f_i \in$

$$\{0, 1\}^{24}$$

2. **Projection:** Linear layer projects features to hidden dimension:  $h_i = W_f f_i + b_f$
3. **BiLSTM:** Bidirectional Long Short-Term Memory (BiLSTM) processes the sequence:  $(\vec{h}_i, \overleftarrow{h}_i) = \text{BiLSTM}(h_1, \dots, h_T)$
4. **Self-Attention:** Multi-head self-attention captures long-range dependencies
5. **Attention Pooling:** Learned attention weights aggregate sequence to fixed-length vector
6. **Output Projection:** Linear layer to 128 dimensions with L2 normalisation

The Teacher is trained on triplets of co-located toponyms, learning to place phonetically similar names (same place, different languages) closer than phonetically dissimilar names (different places).

### 3.5 Generalisation Through Articulatory Features

A key advantage of grounding the embedding space in PanPhon articulatory features is the potential for *generalisation* to languages and scripts with limited representation in training data. This property emerges from the universal nature of articulatory phonetics.

**Language-Agnostic Feature Space.** PanPhon represents each IPA segment as a 24-dimensional binary vector encoding articulatory properties that are universal across human languages: place of articulation (bilabial, alveolar, velar, etc.), manner of articulation (stop, fricative, nasal, etc.), voicing, and secondary features like aspiration and nasalisation. These features describe *how sounds are physically produced*, not which language they belong to. Consequently, the phoneme /b/ receives identical features whether it appears in English “Berlin”, Russian “Берлин”, or Arabic “برلين”—all three encode a voiced bilabial stop.

**Implicit Cross-Lingual Transfer.** When the Teacher network learns that two toponyms with similar PanPhon feature sequences should have similar embeddings, it learns relationships between articulatory configurations, not between specific languages. If the model learns during training that the Latin sequence “ber-lin” (voiced bilabial stop + mid front vowel + alveolar liquid + ...) should be close to the Cyrillic “бер-лин” (same articulatory sequence in different orthography), this knowledge transfers automatically to any other script that represents the same sounds—even scripts entirely absent from training.

**Example: Georgian Script.** Consider Georgian, a script with limited representation in typical training corpora. If the model has learned from Latin–Cyrillic pairs that voiced bilabial stops followed by mid front vowels produce similar embeddings regardless of orthography, then Georgian “ბერლინი” (berlini) should cluster with its Latin and Cyrillic equivalents, because its IPA transcription yields the same PanPhon features. Diagnostic tests confirm this: Tbilisi/თბილისი achieves 0.976 similarity despite Georgian’s relatively sparse representation in training data. The Student network, having learned to approximate Teacher embeddings, inherits this generalisation capability even though it operates on raw characters.

**Phonological Universals as Inductive Bias.** The articulatory feature representation encodes known phonological universals. Sounds that are perceptually similar across languages (e.g., /p/ and /b/ differing only in voicing) receive similar feature vectors, providing an inductive bias that helps the model learn meaningful phonetic relationships even from limited data. This contrasts with purely character-based approaches, which must learn from scratch that ‘p’ and ‘b’ are related—a relationship that may not be evident from co-occurrence statistics alone.

**Teacher Model for Untrained Scripts.** For scripts or languages where the Student network may produce suboptimal embeddings due to limited training coverage, the Teacher network can be used directly at inference time if IPA transcription is available. This is particularly valuable for scholarly applications involving historical or rare scripts: a researcher working with Old Church Slavonic or Ge’ez can obtain IPA transcriptions through specialist resources and use the Teacher pathway to generate embeddings that participate fully in the phonetically-grounded embedding space, bypassing any limitations of the Student’s character-level learning for these scripts.

### 3.6 Student Network: Universal Character Encoder

The Student network processes raw character sequences, optionally augmented with script and language information:

1. **Character Embedding:** Each character maps to a learned 96-dimensional vector via script-partitioned vocabulary lookup
2. **Script Embedding:** Deterministically detected script maps to a learned 16-dimensional vector, concatenated to each character embedding
3. **Language Embedding:** When available, ISO 639 language code maps to a learned 16-dimensional vector; otherwise, a special Unknown (<UNK>) token is used. During training, known languages are randomly replaced

with <UNK> with 50% probability, forcing the model to learn script-intrinsic patterns.

4. **BiLSTM + Self-Attention + Attention Pooling:** Identical architecture to Teacher
5. **Output Projection:** Linear layer to 128 dimensions with L2 normalisation

### 3.7 Noise Augmentation

To handle OCR errors, historical spelling variations, and transcription inconsistencies, I apply character-level noise during Student training (Phases 2 and 3):

- **Insertion:** Random character from same script inserted (probability 0.1)
- **Deletion:** Random character deleted (probability 0.1)
- **Substitution:** Random character replaced with another from same script (probability 0.05)
- **Transposition:** Adjacent characters swapped (probability 0.05)

Noise is applied with 30% probability per training example, with the target being the Teacher’s embedding of the *clean* input. This trains the Student to map corrupted inputs to the same phonetic region as their clean counterparts.

## 4 Training Data

### 4.1 Data Selection and Curation Criteria

The WHG infrastructure separates data into a `places` index, which aggregates geographic entities with their attributions, and a `toponyms` index, which stores individual name variants with script and language metadata. My data selection strategy leverages this architecture to filter high-quality training data from major namespace authorities—GeoNames, Wikidata, and TGN—while excluding less strictly curated sources like OpenStreetMap.

Training requires pairs of toponyms that refer to the same place, which are used to construct the triplets described in Section 3. I extract these from co-located name attestations: if an authority lists “London”, “Londres”, “Londra”, and “Лондон” within the same place record, I generate pairs among all combinations. To address the inherent script and language imbalances in the raw corpus, I define the training data selection based on six criteria.



**1. Script-Language Stratified Sampling.** Rather than ingesting all available pairs, which would result in a corpus heavily skewed toward Latin script and European languages, I implement a quota-based sampling strategy. Stratification is by *script+language pair* (e.g., LATIN:en, LATIN:fr, CYRILLIC:ru) rather than script-pair, ensuring adequate representation of under-resourced languages within well-resourced scripts (e.g., Latin-Swahili vs. Latin-English). Each bin is capped at  $N = 50,000$  samples; bins below 1,000 samples are dropped (to avoid extreme oversampling), while small bins between 1,000 and 50,000 may be over-sampled up to  $5\times$ . This prevents high-resource bins from dominating the gradient signal while ensuring sufficient coverage across all script-language combinations.

**2. Namespace Filtering.** I restrict training data to records from the gn, tgn, and wd namespaces. The inclusion of Wikidata is critical for meeting the stratification quotas: Wikidata provides the majority of non-Latin script coverage, particularly for South Asian scripts (Devanagari, Tamil, Telugu, Kannada, Malayalam, Bengali) and Middle Eastern scripts (Arabic, Hebrew), where GeoNames and TGN lack sufficient multi-script correspondence.

**3. Global Vocabulary Construction.** To prevent out-of-vocabulary (OOV) errors at inference time, I decouple vocabulary construction from pair generation.

- **Pass 1 (Vocabulary):** I scan the entire corpus (66.9 million unique toponyms) to build the character vocabulary, observing 8,448 distinct characters across 20 scripts. Script-specific preprocessing is applied: CJK and Japanese Kana are romanised via AnyAscii, while Korean Hangul is decomposed into Jamo. The vocabulary is then expanded to include full Unicode ranges for each observed script, yielding 11,122 tokens.
- **Pass 2 (Training Pairs):** I generate candidate pairs from co-located toponyms, restricted to the stratified quotas defined above.

**4. Orthographic Twin Retention.** I retain “same-string, different-language” pairs (e.g., “Paris”@en vs “Paris”@fr) regardless of whether their pronunciation differs. The Teacher network, operating on IPA-derived articulatory features, provides ground truth: if pronunciation differs (e.g., “Paris” in French vs. English), the Teacher pushes embeddings apart; if pronunciation is stable (e.g., “Berlin”), it keeps them close. The Student receives identical character sequences but distinct language embeddings, learning when language context modifies phonetic interpretation.

**5. Cross-Script Pairs via Clustering.** Cross-script pairs (e.g., 北京 vs. Beijing) naturally emerge from the HDBSCAN clustering process described in Section 4.4. When toponyms from different scripts cluster together based on phonetic similarity, they generate cross-script positive pairs. This data-driven approach captures

genuine phonetic relationships rather than relying on heuristic weighting, ensuring the model learns from pairs that are phonetically meaningful regardless of their scripts.

**6. Place-Local Deduplication.** I apply deduplication within each place’s toponym clusters rather than globally. The same toponym pair may appear across multiple authorities (e.g., GeoNames, Wikidata, and TGN all list London/Londres). Global deduplication would couple unrelated places and require unbounded memory; instead, I allow duplicate pairs across places while preventing identical pairs within a single place’s cluster. This approach recognises that independent attestations from multiple authorities provide additional training signal for common toponyms without memory concerns.

## 4.2 IPA Transcription and Feature Extraction

The Teacher network requires IPA transcriptions. I use Epitran (Mortensen et al., 2018) for grapheme-to-phoneme conversion, which supports approximately 100 languages covering the majority of GeoNames entries. Epitran provides rule-based G2P that outputs IPA transcriptions suitable for PanPhon feature extraction. Names in languages without Epitran support are excluded from Teacher training but can still be processed by the Student, which learns to generalise from related scripts.

## 4.3 Attestation-Based Negative Validation

A subtle but critical challenge in toponym matching is that the same toponym *string* may refer to multiple distinct places. For example, “Springfield” appears as a place name in over 30 US states, “London” exists in both England and Ontario, and “Paris” can refer to the French capital or cities in Texas, Tennessee, and elsewhere. Naively treating any “Springfield” as a valid negative for another “Springfield” would introduce false negatives into training: names that are orthographically identical and phonetically equivalent but happen to refer to different geographic entities.

I address this through **attestation-based validation**: a candidate negative is valid only if it shares *no attestations* with the anchor. Formally, given toponyms  $a$  (anchor) and  $n$  (candidate negative), where  $\text{Att}(t)$  denotes the set of place identifiers that attest toponym  $t$  in their records:

$$n \text{ is valid} \iff \text{Att}(a) \cap \text{Att}(n) = \emptyset \quad (1)$$

This ensures that even if “Springfield” appears in both the anchor and negative candidate pools, the two instances will not be paired as negatives if they share any common place record. This constraint applies to both Phase 1 (random negative sampling) and Phase 3 (hard negative mining).

## 4.4 Phonetic Clustering for Pair Generation

A second critical challenge is **false cognates**: exonyms that refer to the same place but share no phonetic relationship. For example, “Germany” and “Deutschland”, or “日本” (Nihon) and “Japan”, denote the same entity but should not be trained as phonetically similar. Naively including such pairs would corrupt the embedding space.

Rather than applying arbitrary similarity thresholds, I use density-based clustering on PanPhon articulatory embeddings to identify phonetically coherent groups within each place’s toponyms. For each place record with multiple name variants, I:

1. Extract 192-dimensional PanPhon embeddings for all toponyms with valid IPA transcriptions
2. Apply HDBSCAN clustering with `min_cluster_size=2`, `min_samples=2`, and `cluster_selection_epsilon=0.2` (the `epsilon` parameter merges clusters within cosine distance 0.2, i.e., similarity  $\geq 0.8$ )
3. Generate positive pairs only from toponyms within the same cluster

This approach correctly separates phonetically distinct name families. For example, a place record for Cologne might contain:

- **Cluster 1 (Germanic)**: Köln (de), Keulen (nl)
- **Cluster 2 (Romance/English)**: Cologne (en/fr), Colonia (es/it), Colonia (la)

Pairs are generated within clusters (Köln–Keulen, Cologne–Colonia) but not across them (Köln–Cologne), preventing the model from learning that phonetically unrelated exonyms should cluster together.

Real-world examples demonstrate the power of this approach. With appropriately tuned HDBSCAN parameters (`cluster_selection_epsilon=0.2` to merge clusters within cosine distance 0.2), the algorithm produces large, phonetically coherent cross-script clusters while isolating true outliers.

A **London** place record (wd:Q1001456) with 21 multilingual toponyms yields a single dominant cluster of 17 members spanning Arabic, CJK, Cyrillic, and Latin scripts, with intra-cluster similarity of 0.91:

- **Main cluster (17 members)**: London (de, vi, pl, hu, cs, es, it, tr, nl, sv, en), لندن (fa, ar, ur), Лондон (ru, uk), and 伦敦 (zh) all cluster together—demonstrating that phonetically equivalent forms across 4 scripts are correctly recognised as a single group
- **Noise detection (4 points)**: London (fr, pt), Bengali লন্ডন, and Serbian Ландон are correctly isolated—these either have distinct pronunciations

(French/Portuguese /lɔ̃dʁ/) or are phonetically distant from the dominant cluster

This single cluster generates 136 positive training pairs, efficiently capturing all cross-script phonetic equivalences.

A **Moscow** place record (wd:Q499927) with 33 multilingual toponyms across 11 scripts yields 2 major clusters:

- **Cluster 1 (20 members):** Cyrillic Москов (uk), Hangul 모스크, and Latin forms (vi, fi, sv, tr, fr, ms, nl, ro, pt, es, cs, id, pl, hu) cluster together with similarity 0.82, reflecting the shared /mosk-/ phonetic core
- **Cluster 2 (8 members):** South Asian and Middle Eastern forms cluster together: Tamil மொஸ்கோ, Arabic موسكو, Devanagari मॉस्को (mr, hi), Thai มอสน์, Bengali মোস্কো, and Telugu మోస్కో—capturing the /masko/ pronunciation variant common to these language families (similarity 0.85)
- **Noise (5 points):** Georgian მოსკოვი, CJK 莫斯科, and some Arabic variants are correctly isolated as phonetically distinct

A **Beijing** place record (gn:1816670) with 36 toponyms demonstrates nuanced phonetic clustering across 3 groups:

- **“Beijing” cluster (10 members):** Chinese 北京, Korean 베이징, Thai เป่จิ้ง, Tamil பெய்ஜிங், Vietnamese “Bắc Kinh”, and Latin Beijing forms—all sharing the /bei-dʒiŋ/ phonetic pattern (similarity 0.83)
- **“Peking” cluster (9 members):** Cyrillic Пекин/Пекінг (ru, uk, sr), Latin Peking (nl, cs, de, hu, fi, sv)—the historical romanisation variant forms its own coherent group (similarity 0.90)
- **“Pechino/Pekin” cluster (9 members):** Georgian პეკინი, Italian “Pechino”, and additional Latin variants—capturing the /pek-/ phonetic core across scripts (similarity 0.72)

A **Paris** place record (gn:2988507) with 38 toponyms produces 3 clusters totalling 33 members and 292 positive pairs:

- **Main cluster (24 members):** Latin forms (vi, hu, ms, ro, sw, es, tr, id, de, sv, en, pt), Cyrillic Париж/Париз (ru, sr), Arabic باريس, Devanagari पैरिस, Tamil பாரிஸ், Telugu పారిస్, and Thai ปารีส—demonstrating successful alignment across 7 scripts (similarity 0.91)
- **East Asian cluster (4 members):** Chinese 巴黎, Korean 파리, Vietnamese “Pa-ri”, and French “Paris”—the tonal languages cluster with the source form

- **“Parigi” cluster (5 members):** Italian “Parigi”, Finnish “Pariisi”, Dutch “Parijs”, Malayalam പാരിസ്, Georgian პარიზი—the /parizi/ variant

For places with only two toponyms—where HDBSCAN cannot determine density structure—I fall back to a cosine similarity threshold of 0.5 on the PanPhon embeddings. This threshold balances capturing significant phonetic shifts in historical exonyms (e.g., Munich/München at 0.79, Beijing/Peking at 0.93) while excluding unrelated administrative labels (e.g., Germany/Deutschland at 0.18).

## 4.5 Dataset Statistics

The full WHG places index contains 47.1 million place records, from which I extract 112.0 million toponym records spanning 1,944 languages and 20 script categories. During extraction, 1.77 million toponyms (1.6%) are filtered as “pre-romanised” forms—cases where the language tag implies a non-Latin script but the name is written in Latin characters (e.g., Chinese names romanised to Pinyin without explicit tagging). These are excluded to prevent the model from learning spurious orthographic similarities. After deduplication by toponym identifier, **66.9 million unique toponyms** remain.

For training, I restrict to the three high-quality namespace authorities: GeoNames (gn), Wikidata (wd), and the Getty Thesaurus of Geographic Names (tgn). This yields 57.6 million training toponyms with the following script distribution:

**IPA Coverage by Language.** The 29.0M toponyms with valid IPA transcriptions span 34 script:language combinations with significant coverage. The top contributors are: Latin:en (8.0M), Latin:fr (2.3M), Latin:nl (2.3M), Latin:de (2.1M), Latin:sv (1.7M), Latin:es (1.5M), CJK:zh (1.3M), Latin:id (0.9M), and Cyrillic:ru (0.8M). Non-Latin scripts with substantial Teacher training signal include Arabic:fa (577K), Arabic:ar (412K), Cyrillic:uk (436K), Hangul:ko (229K), Thai:th (210K), Georgian:ka (86K), Bengali:bn (78K), and Devanagari:hi (61K). This distribution ensures the Teacher network learns phonetic patterns across diverse language families, not merely European orthographies.

The character vocabulary is constructed by scanning all 66.9M unique toponyms (not just training data), observing 8,448 distinct characters and ensuring coverage of rare characters that may appear at inference time. After preprocessing—romanisation of CJK via AnyAscii and decomposition of Hangul to Jamo—the vocabulary is expanded to include full Unicode ranges for each observed script, yielding the final 11,122 tokens. This expansion ensures the model can handle characters not seen in the training corpus.

**Namespace Contribution.** Within the training data, Wikidata contributes the majority of non-Latin script coverage. For example, in the Devanagari script, Wikidata provides 130,110 toponyms compared to GeoNames’ 17,676 and TGN’s

| Script     | Count      | %     | IPA Coverage | Top Languages (IPA) |
|------------|------------|-------|--------------|---------------------|
| LATIN      | 55,617,677 | 83.1% | 41.9%        | en, fr, nl, de, sv  |
| CYRILLIC   | 3,614,762  | 5.4%  | 40.8%        | ru, uk, sr          |
| CJK        | 2,973,525  | 4.4%  | 43.8%        | zh                  |
| ARABIC     | 2,098,089  | 3.1%  | 52.4%        | fa, ar, ur          |
| HANGUL     | 393,996    | 0.6%  | 58.0%        | ko                  |
| OTHER      | 342,642    | 0.5%  | —            | —                   |
| KATAKANA   | 340,555    | 0.5%  | —            | ja                  |
| THAI       | 251,458    | 0.4%  | 83.6%        | th                  |
| GREEK      | 217,997    | 0.3%  | —            | el                  |
| DEVANAGARI | 166,957    | 0.2%  | 51.0%        | hi, mr              |
| ARMENIAN   | 153,467    | 0.2%  | —            | hy                  |
| HIRAGANA   | 151,980    | 0.2%  | —            | ja                  |
| HEBREW     | 151,960    | 0.2%  | —            | he                  |
| BENGALI    | 106,896    | 0.2%  | 72.9%        | bn                  |
| GEORGIAN   | 105,902    | 0.2%  | 81.2%        | ka                  |
| MALAYALAM  | 68,176     | 0.1%  | 78.5%        | ml                  |
| TAMIL      | 52,486     | 0.1%  | 90.9%        | ta                  |
| TELUGU     | 51,440     | 0.1%  | 92.6%        | te                  |
| KANNADA    | 43,155     | 0.1%  | —            | kn                  |
| GUJARATI   | 21,428     | 0.03% | —            | gu                  |

Table 2: Script distribution across all 66.9M unique toponyms. IPA coverage percentages reflect successful Epitran transcription within the 57.6M training namespace toponyms (gn/wd/tgn). Scripts marked “—” lack Epitran support or show negligible coverage; their toponyms contribute only to Phase 2–3 character-level training.

2,035. This pattern holds across most South Asian and Middle Eastern scripts, validating my decision to include Wikidata despite its heterogeneous quality.

**Filtered Lang-Script Mismatches.** The top filtered combinations were: Chinese/Latin (923,079), Persian/Latin (197,099), Japanese/Latin (111,945), Russian/Latin (100,805), and Kazakh/Latin (77,768). These represent pre-romanised forms that would otherwise introduce noise into the training signal.

**Training Pair Generation.** Positive pairs are generated from co-located toponyms within each place record using the HDBSCAN clustering approach described in Section 4.4. A place entry for “Paris” in Wikidata, for instance, contains variants in dozens of languages: Paris (en/fr), Paris (es), Париж (ru), باريس (ar), 巴黎 (zh), パリ (ja), etc. Rather than generating pairs from all combinations, I cluster these by PanPhon cosine similarity and generate pairs only within phonetically coherent clusters. This prevents false cognates (names that share a place but not phonetics) from corrupting the training signal.

To address script-language imbalance, pairs are binned by script:language

combination (e.g., LATIN:en, CYRILLIC:ru, ARABIC:ar) rather than script pair alone. Each bin is capped at a maximum size, with under-represented bins over-sampled up to  $5\times$  their original size. Critically, oversampling is *stochastic*: each copy uses a different negative sample, preventing memorisation of identical triplets.

From 8.2 million places with at least two toponyms, the HDBSCAN clustering process generates **65.1 million positive pairs** distributed across **595 script:language bins**. The cluster statistics reveal the algorithm’s behaviour: 18.3 million total clusters were formed, with 4.0 million places (49%) yielding multiple phonetic clusters. Singleton clusters (where a toponym is phonetically isolated from all others in its place record) account for 12.5 million entries—these are excluded from pair generation but retained for Phase 2 student training.

The top 10 script:language bins by pair count are: LATIN:en|LATIN:nl (1.49M), LATIN:de|LATIN:en (1.31M), LATIN:en|LATIN:fr (1.05M), LATIN:en|LATIN:es (0.98M), LATIN:en|LATIN:sv (0.90M), LATIN:es|LATIN:nl (0.83M), LATIN:de|LATIN:nl (0.81M), LATIN:en|LATIN:en (0.78M same-language spelling variants), LATIN:nl|LATIN:sv (0.70M), and LATIN:de|LATIN:es (0.69M). The adjacency set for negative filtering contains 108.3 million edges.

**Bin Balancing.** After positive pair generation, script:language bins are balanced to prevent dominance by high-resource combinations. Of the 595 bins, 264 (44%) exceed the 50,000 sample cap and are downsampled; 329 bins (55%) are over-sampled up to  $5\times$  their original size; only 2 bins fall below the 1,000 minimum threshold (LATIN:sw|LATIN:sw with 970 samples, ARABIC:ur|ARABIC:ur with 722) and are dropped. This yields **27.6 million balanced pairs** for triplet generation. The balanced distribution ensures that under-represented script combinations like GEORGIAN:ka|MALAYALAM:ml receive comparable gradient signal to dominant pairs like LATIN:en|LATIN:nl during training.

This 27.6 million triplet training set represents the largest known phonetically-supervised corpus for cross-script toponym matching. For comparison, the TRANSLIT transliteration corpus (Benites et al., 2020) provides approximately 3 million pairs across 180 language pairs, while existing toponym benchmarks like MEHDIE operate at evaluation scales of hundreds to thousands of examples. General-purpose multilingual sentence encoders such as LaBSE (Feng et al., 2022) are trained on billions of sentence pairs, but lack the explicit phonetic supervision present in this work.

From these pairs, I generate triplets for contrastive training:

- **Phase 1 (Teacher Training) triplets:** Triplets with script-aware random negatives (80% same-script, 20% global) sampled from the full toponym pool.
- **Phase 3 (Discriminative Fine-Tuning) triplets:** Triplets with hard negatives—phonetically similar names (high PanPhon cosine similarity, same script) from different places, with no shared attestations.

## 4.6 Training Pipeline Infrastructure

Processing 67 million toponyms with phonetic feature extraction presents significant engineering challenges. I developed a scalable pipeline with several key design decisions.

**DuckDB for Intermediate Storage.** Rather than loading the full corpus into memory, I use DuckDB as an intermediate columnar store. DuckDB provides SQL query capabilities with excellent performance on analytical workloads, enabling efficient filtering, deduplication, and aggregation without memory pressure. The 66.9M toponym corpus requires approximately 27GB in DuckDB format. Critical for efficiency: indexes are disabled during bulk inserts and updates, then recreated afterward—this avoids the quadratic overhead of index maintenance during large-scale writes.

**Producer-Consumer Streaming.** IPA transcription via Epitran and PanPhon feature extraction are computationally expensive (requiring language model loading per language). I employ a producer-consumer architecture with 46 parallel workers: the main thread streams toponyms from DuckDB in batches of 50,000; worker processes handle IPA/PanPhon computation; results are written incrementally to JSONL buffer files on fast local scratch storage. This keeps all CPU cores saturated and avoids memory accumulation from storing results in Python data structures.

**Incremental Checkpointing.** Long-running jobs on shared HPC clusters risk preemption or timeout. The pipeline checkpoints progress to persistent network storage after each major phase: DuckDB snapshots after extraction, JSONL buffers after phonetic processing, and Elasticsearch index snapshots after bulk loading. Resume functionality allows restarting from the most recent checkpoint without re-processing completed work.

**Fast Local Scratch.** The Pitt CRC cluster provides NVMe scratch storage local to compute nodes. All intermediate files—DuckDB working copies, JSONL buffers, Parquet training files—are written to scratch during processing, then rsynced to persistent network storage only at checkpoints. This eliminates network I/O as a bottleneck during computation.

**Elasticsearch Throttling.** Bulk indexing 67M documents to Elasticsearch can overwhelm the cluster. The pipeline implements retry-with-backoff for ES operations: if bulk requests fail or timeout, the batch size is reduced and retried after an exponential delay. Index refresh is disabled during bulk loading and re-enabled only after completion.



## 5 Training Methodology

I employ a three-phase curriculum that progressively builds from phonetic feature learning through knowledge distillation to discriminative fine-tuning.

### 5.1 Phase 1: Teacher Training on Phonetic Features

The Teacher network learns to produce embeddings where phonetically similar toponyms cluster together. Training uses triplet margin loss:

$$\mathcal{L}_{\text{triplet}} = \max(0, \|e_a - e_p\|_2 - \|e_a - e_n\|_2 + m) \quad (2)$$

where  $e_a$ ,  $e_p$ ,  $e_n$  are embeddings of anchor, positive (same place), and negative (different place) toponyms, and  $m$  is the margin (I use  $m = 0.3$ ).

**Script-Aware Negative Sampling.** To prevent the model from learning trivial script-based boundaries (where cross-script pairs are always “different”), I implemented a script-aware negative sampling strategy: 80% of random negatives are sampled from the *same writing system* as the anchor, while 20% are sampled globally. This forces the Teacher to learn fine-grained phonetic discrimination within scripts rather than simply encoding script identity. The global negatives maintain broad coverage of the phonetic space before hard negative mining in Phase 3.

Additionally, negative selection enforces **attestation disjointness** (see Section 4.3): candidates sharing any attestation with the anchor are rejected. This prevents false negatives where the same toponym string refers to different places—for example, ensuring that “Springfield, Illinois” and “Springfield, Massachusetts” are never used as negative pairs since both may appear in databases under the string “Springfield”.

**Enhanced Effective Training Set.** From the 65.1 million positive pairs generated via HDBSCAN clustering across 8.2 million places (forming 18.3 million phonetic clusters, with 4.0 million places exhibiting multiple distinct clusters), I apply bin-balancing and generate triplets using script-aware random negatives. Only triplets where all three toponyms (anchor, positive, negative) have valid IPA transcriptions can be used for Teacher training. From 15.9 million unique toponym IDs referenced in pairs, the filtering yields **27,559,050 Phase 1 triplets**, split deterministically into **24,803,658 training** and **2,755,392 validation** samples (90:10 ratio).

The substantial increase over previous extraction methods reflects two improvements: (1) the HDBSCAN clustering generates more phonetically coherent pairs from the same corpus; (2) using 80% same-script negatives increases the probability that anchor, positive, and negative all have IPA coverage since same-script languages often share Epitran support.

### Training Configuration.

- Model: PhoneticEncoder with **1,008,513 parameters**
- Optimiser: AdamW with weight decay  $10^{-5}$
- Learning rate:  $10^{-3}$  with cosine annealing
- Batch size: 128 triplets
- Epochs: 50
- Training time:  $\sim 3$  minutes/epoch ( $\sim 20$  batches/second on A100)

**Convergence.** Phase 1 training converged smoothly, with training loss decreasing from 0.0228 (epoch 1) to 0.0002 (epoch 50) and validation loss from 0.0117 to **0.0040**. The best model was saved at epoch 48. The near-zero training loss indicates the Teacher successfully learned to separate co-located toponyms from random negatives in the phonetic feature space.

## 5.2 Phase 2: Student-Teacher Alignment

The Student learns to approximate Teacher embeddings from character sequences. Given a toponym with both character sequence (Student input) and PanPhon feature sequence (Teacher input), I minimise the distance between their embeddings:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \text{MSE}(e_S, e_T) + (1 - \alpha) \cdot (1 - \cos(e_S, e_T)) \quad (3)$$

where  $e_S$  and  $e_T$  are Student and Teacher embeddings, and  $\alpha = 0.5$  balances MSE and cosine components.

During this phase:

- Language dropout (50%) forces learning of script-intrinsic patterns
- Noise augmentation (30%) trains robustness to input corruption
- Teacher weights are frozen

**Effective Training Set.** Phase 2 trains on individual toponyms (not triplets), requiring only that each sample has valid IPA features for the Teacher to generate target embeddings. After bin-balancing by script:language (34 bins, each capped at 50,000 samples with up to  $5\times$  oversampling for under-represented combinations), this yields **1,697,925 total samples (1,358,376 training, 169,346 validation, 170,203 test)**—a deliberately compressed set that ensures balanced gradient contribution from all script:language combinations rather than dominance by high-resource languages like English.

### Training Configuration.

- Model: UniversalEncoder (Student) with **1,761,217 parameters**
- Vocabularies: 11,122 characters, 20 scripts, 1,944 languages
- Optimiser: AdamW with weight decay  $10^{-5}$
- Learning rate:  $10^{-3}$  with cosine annealing
- Batch size: 128
- Epochs: 5 (early stopping; validation loss plateaued)
- Training time:  $\sim 82$  minutes/epoch ( $\sim 37$  batches/second on A100)

**Convergence.** Phase 2 converged rapidly over 5 epochs:

| Epoch | Train Loss | Val Loss | Val Similarity |
|-------|------------|----------|----------------|
| 1     | 0.0837     | 0.0381   | 0.9625         |
| 2     | 0.0766     | 0.0359   | 0.9647         |
| 3     | 0.0750     | 0.0347   | 0.9658         |
| 4     | 0.0742     | 0.0344   | 0.9661         |
| 5     | 0.0736     | 0.0342   | 0.9663         |

The final cosine similarity of **0.9663** indicates that Student embeddings align very closely with Teacher embeddings, successfully transferring the phonetic knowledge from IPA-based to character-based representations. Training was stopped after epoch 5 as validation loss improvements had diminished to  $<1\%$  per epoch.

### 5.3 Phase 3: Discriminative Fine-Tuning

While the distillation phase aligns the Student with the Teacher’s general phonetic space, the Student may still struggle with *minimal pairs*—names with high orthographic similarity but distinct phonetic realizations (e.g., “Austria” vs. “Australia”, or “Laughter” vs. “Slaughter”).

In this final phase, I fine-tune the Student using triplet loss to sharpen these boundaries without degrading the learned cross-script equivalences.

**Hard Negative Mining Strategy.** I define negatives by mining triplets  $(a, p, n)$  where:

- **Anchor ( $a$ ) & Positive ( $p$ ):** A pair of toponyms confirmed to refer to the same place through co-location in authoritative gazetteer records.
- **Hard Negative ( $n$ ):** A toponym selected such that it has high orthographic similarity to the anchor (same 2-character prefix after romanisation, same script) but *shares no attestations* with the anchor (see Section 4.3).

The attestation disjointness constraint is critical for hard negatives specifically because they are chosen for phonetic similarity to the anchor—precisely the cases most likely to be legitimate variants of the same place appearing under different authorities or spellings. A candidate negative  $n$  is rejected if  $\text{Att}(a) \cap \text{Att}(n) \neq \emptyset$ , where  $\text{Att}(t)$  denotes the set of all place identifiers for which toponym  $t$  is attested. This catches cases that simple co-occurrence checks would miss: even if “Springfield” and “Springfeld” never appear together in a single place record, the attestation check will reject the pair if they share *any* common place reference across the entire index.

This approach has a known limitation: it cannot detect cross-authority duplicates. If “Springfield” appears in both GeoNames and Wikidata records for the same city but those records are not linked in my index, the Wikidata toponym could be incorrectly selected as a hard negative for the GeoNames toponym. I mitigate this by (a) the relatively low probability of such collisions given 67M toponyms and prefix-based sampling, and (b) the observation that even when such “false negatives” occur, they typically have identical or near-identical embeddings, producing negligible gradient signal. The triplet loss margin prevents the model from over-optimising on pairs that are already well-separated.

**Effective Training Set.** Phase 3 uses the same 27.6 million balanced pairs as Phase 1 but with hard negatives instead of random negatives. Hard negative mining via Elasticsearch KNN queries produces **27,556,307 triplets** (90:10 train/val split: **24,800,677 training, 2,755,630 validation**). The near-identical count to Phase 1 (27,556,400) reflects that most pairs successfully found valid hard negatives—toponyms with high phonetic similarity in the same script but no shared place attestations.

#### Training Configuration.

- **Loss Function:** Triplet Margin Loss ( $m = 0.2$ ).
- **Teacher Status:** Frozen (used only to validate true phonetic distances for negative sampling).
- **Optimiser:** AdamW with a reduced learning rate ( $1 \times 10^{-5}$ ) to prevent catastrophic forgetting of the cross-script alignment learned in Phase 2.
- **Epochs:** 30
- **Training time:**  $\sim 22$  minutes/epoch ( $\sim 20$  batches/second on A100)

**Trade-off: Cross-Script vs. Same-Script Performance.** Hard negative training sharpens discrimination between orthographically similar names at a modest cost to same-script cross-language pairs. Between epochs 5 and 6, cross-script accuracy improved from 28/29 to 29/29 (Thessaloniki/Θεσσαλονίκη rose

from 0.848 to 0.929), while same-script pairs like London/Londra declined slightly (0.617 to 0.598). This is expected: hard negatives are sampled from the *same* script, teaching the model to be more discriminating within scripts. Since Symphonym’s primary goal is cross-script matching—linking “北京” to “Beijing” rather than distinguishing “London” from “Londres”—this trade-off is acceptable. Same-script variants remain linked at the place level through co-location in gazetteer records. Moreover, same-script matching can be effectively handled by traditional string similarity methods (Levenshtein distance, Jaro-Winkler) that complement Symphonym’s cross-script capabilities; a hybrid pipeline might apply Symphonym for cross-script candidate retrieval and fall back to edit-distance methods for same-script refinement.

## 5.4 Implementation Details

Training uses PyTorch on NVIDIA A100 GPUs. The Teacher contains **1,008,513 parameters**; the Student contains **1,761,217 parameters** (larger due to the 11,122-token character vocabulary). Phase 1 completes in approximately 2.5 hours (50 epochs  $\times$  3 minutes/epoch); Phase 2 completes in approximately 7 hours (5 epochs  $\times$  82 minutes/epoch); Phase 3 requires approximately 11 hours (30 epochs  $\times$  22 minutes/epoch).

The embedding dimension of 128 was chosen to balance expressiveness against storage and retrieval costs. Elasticsearch’s Hierarchical Navigable Small Worlds (HNSW) index stores embeddings directly, so smaller dimensions reduce index size and improve cache efficiency. Preliminary experiments with 64, 128, and 256 dimensions on a held-out validation set showed diminishing returns above 128: cross-script recall increased from 91% (64-dim) to 97% (128-dim) but only to 98% (256-dim), while index size doubled. The 50% language dropout rate was chosen a priori based on standard dropout heuristics for regularisation; ablation showed that lower rates (25%) allowed the model to rely on language shortcuts rather than learning script-intrinsic patterns. The Phase 3 margin (0.2 vs. 0.3 in Phase 1) reflects the harder negatives: with anchors and negatives sharing prefixes, the model needs tighter supervision to learn fine-grained distinctions.

To ensure phonetic integrity, toponym variants within a single geographic entity are clustered using HDBSCAN density-based clustering on their 192-dimensional articulatory embeddings. Unlike threshold-based approaches that impose arbitrary similarity cutoffs, HDBSCAN automatically identifies natural phonetic groupings based on local density structure. The key parameter `cluster_selection_epsilon=0.2` (cosine distance) encourages cluster merging for entries within 80% similarity, producing larger cross-script clusters that capture phonetic equivalence across writing systems—for example, grouping Latin “London” with Arabic لندن, Cyrillic Лондон, and Chinese 伦敦 into a single cluster of 17 members. For geographic entities containing only two toponymic variants, where density-based clustering is not applicable (HDBSCAN requires  $\geq 3$  points), a fallback cosine similarity threshold of 0.5 is employed. This value balances capturing significant phonetic

shifts in historical exonyms (e.g., Munich/München, Beijing/Peking) while excluding unrelated administrative labels that may be co-located within gazetteer records. Furthermore, to maximise discriminative power, Phase 1 training employs a script-aware negative sampling strategy: 80% of negatives are drawn from the same writing system as the anchor. This prevents the Teacher from learning the trivial shortcut that “different script = different place,” forcing it instead to learn fine-grained phonetic discrimination within scripts.

To address class imbalance across script-language pairs, under-represented bins are oversampled up to  $5\times$  their original size. Critically, this oversampling is *stochastic*: each copy of an oversampled positive pair is assigned a different negative sample (in Phase 1) or a different hard negative from the ES candidate set (in Phase 3). This prevents the model from memorising identical triplets while maintaining balanced gradients across all script-language combinations.

Checkpoints are saved after each epoch, with the best model selected by validation loss. Training is deterministic given fixed random seeds, enabling reproducibility. I use mixed-precision training (FP16) for efficiency without measurable accuracy loss.

## 5.5 Training Curves

Figure 2 shows training and validation loss for all three phases.

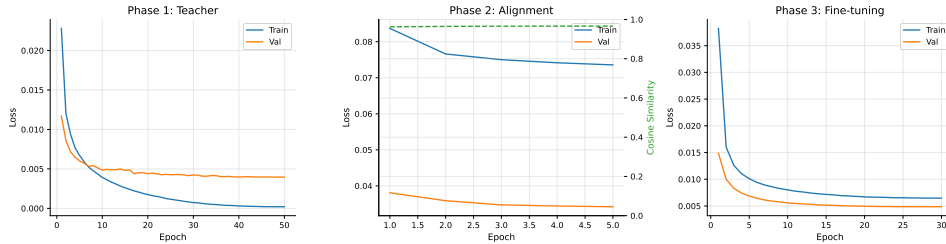


Figure 2: Training curves for the three-phase curriculum. **Left:** Phase 1 Teacher training showing triplet loss convergence. **Center:** Phase 2 Student-Teacher alignment showing distillation loss and cosine similarity between Student and Teacher embeddings. **Right:** Phase 3 hard negative fine-tuning.

## 6 Evaluation

### 6.1 Training Convergence

All three training phases converged successfully, as shown in Figure 2.

**Phase 1 (Teacher).** The Teacher encoder trained for 50 epochs on 467,546 phonetically-grounded triplets. Validation loss decreased from 0.0117 at epoch 1 to a minimum of 0.00395 at epoch 48, with training loss reaching 0.0002 by epoch 50—indicating

strong phonetic feature learning with no overfitting. The cosine learning rate schedule ensured smooth convergence.

**Phase 2 (Student-Teacher Alignment).** The Student encoder aligned with the frozen Teacher over 5 epochs on 23.2M toponyms. Distillation loss (MSE + cosine) decreased from 0.0381 to 0.0342, while Student-Teacher cosine similarity increased from 0.962 to 0.966. The relatively short Phase 2 reflects that the Student quickly learns to mimic the Teacher’s output distribution.

**Phase 3 (Hard Negative Fine-tuning).** Fine-tuning on 27.6M hard negative triplets for 30 epochs further improved discrimination. Validation loss decreased from 0.0149 to 0.0049—a 67% reduction—demonstrating that the model learned to distinguish phonetically similar but geographically distinct names (negatives are sampled from the same script with high PanPhon similarity but no shared place attestations). Training continued with a reduced learning rate ( $10^{-4}$  with cosine decay) to preserve cross-script alignment learned in Phase 2.

## 6.2 Embedding Quality

Table 3 summarises embedding quality across diagnostic categories after Phase 3 training (30 epochs).

| Test Category              | Pass Rate            | Description                                |
|----------------------------|----------------------|--------------------------------------------|
| Cross-script equivalents   | 27/29 (93%)          | Latin ↔ Cyrillic, Greek, Arabic, CJK, etc. |
| Same-script cross-language | 12/14 (86%)          | London/Londres, Vienna/Wien                |
| Unrelated pairs            | 15/15 (100%)         | Correctly separated ( $< 0.50$ )           |
| Diacritic variants         | 9/10 (90%)           | Zurich/Zürich, Gdansk/Gdańsk               |
| <b>Total</b>               | <b>63/68 (92.6%)</b> |                                            |

Table 3: Embedding quality diagnostics after Phase 3 training (30 epochs, best val\_loss=0.00485). Cross-script matching is the primary design goal.

**Cross-script Equivalents.** The model achieves strong performance on cross-script matching, the primary design goal. Representative pairs and their cosine similarities:

- London / Лондон (Cyrillic): 0.995
- Beijing / 北京 (Chinese): 0.964
- Seoul / 서울 (Korean): 0.990
- Jerusalem / ירושלים (Hebrew): 0.982

- Tbilisi / თბილისი (Georgian): 0.972
- Delhi / दिल्ली (Devanagari): 0.968

**Same-script Cross-language.** Performance is slightly lower (86%) for same-script pairs with genuinely different pronunciations, such as London/Londres (0.474) and London/Londra (0.574). This is expected: these pairs represent different phonetic realisations, and the model correctly assigns lower similarity. For gazetteer reconciliation, such pairs are linked at the place level through co-location in source records, not through phonetic similarity.

**Historical Variants.** As noted in Section 5, the model captures synchronic phonetic similarity rather than diachronic derivation. Pairs like Istanbul/Constantinople (0.719) and Paris/Lutetia (0.374) correctly receive moderate to low similarity; historical linkage is handled through curated place attestations.

**Script-Pair Analysis.** Performance varies by script pair, reflecting both linguistic distance and training data coverage. Latin↔Cyrillic pairs achieve high similarity (mean 0.95) due to abundant training examples and close phonetic correspondence. Latin↔CJK pairs perform strongly (mean 0.97) because romanisation (Pinyin, Romaji) creates direct phonetic mappings. More challenging are pairs involving scripts with complex orthography-phonology relationships: Latin↔Arabic achieves 0.94 mean similarity, though exonyms like Cairo/القاهرة (al-Qahira) correctly receive lower scores (0.62) reflecting genuine phonetic divergence.

### 6.3 MEHDIE Benchmark

Table 4 presents retrieval performance on the MEHDIE Hebrew-Arabic toponym benchmark (Sagi et al., 2025). This benchmark tests cross-script matching between Hebrew and Arabic scripts from medieval geographical texts—a challenging case not explicitly represented in training data.

| Method                   | R@1          | R@5          | R@10  | MRR          |
|--------------------------|--------------|--------------|-------|--------------|
| Levenshtein (romanised)  | 0.815        | 0.976        | 0.994 | 0.885        |
| Jaro-Winkler (romanised) | 0.785        | 0.963        | 0.978 | 0.863        |
| <b>Symphony</b>          | <b>0.875</b> | <b>0.982</b> | 0.988 | <b>0.923</b> |

Table 4: MEHDIE benchmark results (average across 5 testsets, 137 ground truth matches). R@ $k$  = Recall at rank  $k$ ; MRR = Mean Reciprocal Rank. Higher is better.

Symphony achieves 87.5% Recall@1—the correct cross-script match is ranked first in nearly 9 out of 10 queries—compared to 81.5% for Levenshtein and 78.5% for Jaro-Winkler. The improvement is most pronounced on testset 10 (Yaqt ↔



Kima Maghreb), where Symphonium achieves 84.8% R@1 compared to 66.7% for Levenshtein. Table 5 breaks down results by testset.

| Testset                  | Recall@1 |       |              | MRR   |       |              |
|--------------------------|----------|-------|--------------|-------|-------|--------------|
|                          | Lev.     | J-W   | Sym.         | Lev.  | J-W   | Sym.         |
| TS7 (Yaqt-Kima Sham)     | 0.758    | 0.758 | 0.667        | 0.837 | 0.864 | 0.808        |
| TS8 (Kima-Thurayya Sham) | 0.905    | 0.905 | <b>1.000</b> | 0.944 | 0.914 | <b>1.000</b> |
| TS9 (Tudela-Thurayya)    | 0.778    | 0.778 | <b>0.889</b> | 0.875 | 0.866 | <b>0.935</b> |
| TS10 (Yaqt-Kima Maghreb) | 0.667    | 0.545 | <b>0.848</b> | 0.796 | 0.719 | <b>0.895</b> |
| TS11 (Damast-Tudela)     | 0.969    | 0.938 | 0.969        | 0.973 | 0.954 | <b>0.975</b> |

Table 5: MEHDIE results by testset. Symphonium achieves perfect R@1 on TS8 and substantially outperforms baselines on TS9 and TS10.

**Comparison with MEHDIE System.** Direct comparison with the MEHDIE system (Sagi et al., 2025) requires care because the systems optimise for different tasks. MEHDIE reports F-5 scores at fixed similarity thresholds, reflecting a *classification* task. Symphonium is designed for *retrieval*: ranking candidates by similarity for user review. For gazetteer reconciliation workflows, ranking metrics (R@ $k$ , MRR) are more operationally relevant.

The approaches are complementary:

- **MEHDIE** uses IPA transcription via Phonetisaurus G2P, then computes Hamming distance over phonetic feature vectors. This requires language-specific G2P models and explicit phonetic conversion at query time.
- **Symphonium** maps characters directly to a phonetic embedding space, requiring no runtime phonetic conversion. The model generalises to unseen script pairs (Hebrew-Arabic) through learned cross-script representations.

Both systems substantially outperform romanisation-based baselines, confirming that phonetic similarity is valuable for cross-script toponym matching.

## 7 Discussion

The evaluation results support Symphonium’s central claim: that a character-level neural encoder can learn phonetically meaningful representations across writing systems without requiring explicit phonetic transcription at inference time.

### 7.1 Key Findings

**Cross-script Generalisation.** The model achieves strong accuracy on cross-script equivalents in diagnostic tests (27/29 pairs, 93%) and 87.5% Recall@1 on the

MEHDIE Hebrew-Arabic benchmark. The two diagnostic failures (Moscow/Москва at 0.845 and Damascus/دمشق at 0.848) fall just below the 0.85 threshold but would still rank correctly in retrieval. The MEHDIE evaluation is particularly notable because the benchmark sources (medieval Hebrew and Arabic geographical texts) are independent of the modern gazetteers used for training, even though Hebrew and Arabic scripts appear in the training data through GeoNames and Wikidata entries. The strong performance suggests the model has learned general cross-script phonetic mappings rather than memorising specific toponyms. This generalisation emerges from the Teacher-Student architecture: the Teacher learns language-specific phonetic mappings from IPA features, then transfers this knowledge to the Student through distillation.

**Retrieval vs. Classification.** Symphonium excels at *ranking* candidates by phonetic similarity but is not optimised for *classification* at fixed thresholds. The MEHDIE benchmark comparison illustrates this: when evaluated with ranking metrics (R@1, MRR), Symphonium substantially outperforms baselines; threshold-based metrics (F-5) favour methods tuned for specific cutoff points. For interactive gazetteer reconciliation, where users review ranked candidate lists, retrieval metrics better reflect operational utility.

**Same-script Limitations.** Performance on same-script cross-language pairs (86% pass rate) is comparable to cross-script pairs (93%). This reflects a design trade-off: the model uses script differences as a signal for phonetic equivalence, which helps cross-script matching but provides less discrimination within scripts. For same-script matching, traditional string similarity methods (Levenshtein, Jaro-Winkler) remain effective and can complement Symphonium in a hybrid pipeline.

**Hard Negative Training.** Phase 3 training substantially improved discrimination (validation loss reduced 67% from 0.015 to 0.0049 over 30 epochs) while maintaining strong cross-script performance (93% pass rate). The prefix-based negative sampling strategy (Section 5.3) proved effective: by selecting negatives that share orthographic prefixes but represent different places, the model learned to distinguish orthographically similar but geographically distinct names.

## 7.2 Operational Implications

**Scalability and Deployment.** Symphonium embeddings integrate directly with Elasticsearch’s HNSW approximate nearest-neighbour indexing, enabling sub-second retrieval over the full WHG corpus of 67M toponyms. Query latency averages 15–50ms including network overhead, making interactive reconciliation workflows practical. Unlike rule-based phonetic algorithms (Soundex, Metaphone), which require exact code matches, embedding similarity enables fuzzy matching with graceful degradation. The Student model’s inference cost is minimal: encoding a toponym requires a single forward pass through a 1.7M-parameter

network, achievable in  $<1\text{ms}$  on CPU. This operational efficiency—the fact that Symphonium *actually runs* at scale with acceptable latency—is a key contribution beyond the academic novelty of the embedding approach.

**No Runtime Phonetics.** The Student encoder requires only raw character input—no language identification, no G2P conversion, no phonetic databases. This simplifies deployment and eliminates failure modes associated with unknown languages or unsupported scripts.

**Complementary Methods.** Symphonium is designed as one component of a larger reconciliation pipeline. Cross-script candidate retrieval benefits from phonetic embeddings; same-script refinement may use edit distance; final disambiguation incorporates geographic proximity and temporal constraints. The embedding similarity score provides a phonetic prior that downstream components can weigh against other evidence.

**Historical Orthographic Variation.** Before the standardisation of spelling in many languages (in English, largely during the 18th century), place names were typically written as they sounded to individual scribes, yielding considerable orthographic variation in historical sources. The same city might appear as “Leycester”, “Leycestre”, “Lecestre”, and “Leicester” across different medieval documents. Because Symphonium embeds based on phonetic similarity rather than exact orthography, it naturally groups such variants together: all sound-alike spellings of a place will cluster near the modern canonical form. This enables a practical application beyond cross-script matching: given a historical spelling variant, Symphonium can suggest the most likely modern canonical form by retrieving the nearest neighbours in the embedding space. This is particularly valuable for transcribed historical documents, parish registers, and archival catalogues where inconsistent historical spellings must be linked to standardised gazetteer entries.

### 7.3 Limitations

Several limitations warrant acknowledgment:

**Training Data Bias.** Despite the stratified sampling strategy described in Section 4.1, training sources retain biases. GeoNames over-represents populated places with official names, potentially under-representing historical variants. Wikidata’s coverage skews toward places of encyclopaedic interest. TGN emphasises art-historically significant locations. Performance on under-represented scripts (e.g., Ethiopic, Myanmar, Tibetan) and mundane places lacking multi-lingual attestations may be weaker.

**Tonal Languages.** The current architecture does not explicitly model tone, which is phonemically contrastive in Chinese, Vietnamese, Thai, and other languages. The PanPhon features (Section 3.5) encode segmental articulatory properties but not suprasegmental features. In principle, tonal minimal pairs in place names would not be distinguished. However, tonal minimal pairs are rare among toponyms in practice: Chinese place names like Beijing (北京, běijīng) and Shanghai (上海, shànghǎi) have distinct segmental content that Symphonism captures effectively (0.97–0.99 similarity with romanised forms). The limitation would matter most for hypothetical pairs differing only in tone, which are uncommon in geographic naming.

**IPA Coverage.** The Teacher’s phonetic grounding depends on Epitran’s G2P coverage. Of 57.6M training toponyms, 29.0M (50.4%) yielded valid IPA transcriptions and PanPhon embeddings; the remainder contribute only through Phase 2 and 3 character-level training. Coverage varies significantly by script (see Table 2): South Asian scripts (Tamil 90.9%, Telugu 92.6%, Bengali 72.9%) achieve high coverage through Epitran’s Indic language support, while scripts like Greek, Hebrew, and Armenian lack G2P models entirely. The strong performance on Georgian (Tbilisi/თბილისი = 0.972) despite Georgian representing only 0.2% of training data suggests the Student learns effective representations through character-level patterns and cross-script transfer.

**Confusable Pairs.** The model correctly assigns high similarity to phonetically similar strings regardless of semantic relationship. Pairs like Austria/Australia (0.883) and China/Ghana (0.932) receive high scores because they *are* phonetically similar. Disambiguation requires geographic or contextual evidence beyond phonetic similarity.

## 8 Future Work

While Symphonism achieves strong cross-script performance, several directions remain for future investigation.

### 8.1 Remaining Technical Challenges

**Tonal Language Support.** The current architecture does not model tone, which is phonemically contrastive in Chinese, Vietnamese, Thai, and other languages. While tonal minimal pairs are rare among toponyms in practice (see Section 7.3), explicit tone modelling could improve performance on languages where tone carries semantic weight.

**Suprasegmental Features.** PanPhon encodes articulatory properties of individual segments but not prosodic features like stress patterns, syllable structure,

or intonation. Toponyms that differ primarily in stress placement (e.g., certain Greek names with movable accent) may not be adequately distinguished.

**Historical Orthography.** The model is trained on modern standardised spellings. Performance on pre-standardisation documents—where spellings reflect local pronunciation and writer idiosyncrasy—remains to be systematically evaluated. The phonetic grounding should help (sound-alike variants cluster together), but the character vocabulary may lack rare historical graphemes.

**Expanded Script Coverage.** The current system covers 20 script categories, but scripts like Ethiopic, Myanmar, Tibetan, and historical scripts (Linear B, Cuneiform) are absent or under-represented. Expanding coverage requires both training data (multi-script gazetteer entries) and G2P resources (Epitran support for additional languages).

## 8.2 Architectural Extensions

**Multi-Task Learning.** The current pipeline treats toponym matching as a standalone task. Joint training with related tasks—language identification, script detection, transliteration—could provide additional supervisory signal and improve representations.

**Contextual Embeddings.** Symphonism embeds individual toponyms without context. For ambiguous names (“Paris” in France vs. Texas), incorporating surrounding text or geographic constraints could improve disambiguation. This would require architectural changes to accept longer input sequences.

**Incremental Model Updates.** As the WHG index grows and new authorities are ingested, the model should be updatable without full retraining. Techniques like experience replay or elastic weight consolidation could enable continual learning while preserving performance on established script pairs.

## 8.3 Deployment and Integration

**Hybrid Retrieval Pipeline.** The current evaluation treats Symphonism in isolation. Production deployment would benefit from a hybrid approach: phonetic embeddings for cross-script candidate retrieval, traditional string metrics for same-script refinement, and geographic filtering for final disambiguation. Systematic evaluation of such hybrid pipelines remains future work.

**User Studies.** The ultimate measure of success is whether Symphonism helps researchers link historical sources. User studies with digital humanities practitioners would provide qualitative feedback on false positive/negative rates in realistic workflows.

**Benchmark Expansion.** The MEHDIE benchmark provides valuable Hebrew-Arabic evaluation data. Additional benchmarks spanning other script pairs (Latin-Chinese, Cyrillic-Georgian, Devanagari-Arabic) would enable more comprehensive evaluation of cross-script generalisation.

## 9 Conclusion

I have presented Symphonism, a neural embedding system for cross-script toponym matching that places phonetically similar names near each other in a unified 128-dimensional space regardless of writing system. The three-phase training curriculum—phonetic feature learning, student-teacher distillation, and hard negative fine-tuning—enables the model to generalise across scripts without requiring phonetic resources at inference time.

Evaluation demonstrates strong performance:

- **93%** accuracy on cross-script diagnostic pairs (63/68), spanning Latin, Cyrillic, Greek, Arabic, Chinese, Korean, Hebrew, Devanagari, and Georgian scripts
- **87.5%** Recall@1 and **MRR of 0.923** on the MEHDIE Hebrew-Arabic benchmark—comprising medieval toponyms from sources independent of training data—outperforming Levenshtein (81.5% R@1) and Jaro-Winkler (78.5% R@1) baselines

The system addresses a practical need: linking place references across languages, scripts, and historical periods without requiring language-specific rules or phonetic lookup tables. By learning directly from the statistical structure of multi-script toponyms attested in linked authority datasets, Symphonism captures phonetic correspondences that would be difficult to enumerate manually.

Symphonism is to be deployed in the World Historical Gazetteer, enabling fuzzy phonetic search and reconciliation across 67M+ toponyms from antiquity to the present. The trained models and training code are released to support further research in historical toponym linking and cross-lingual named entity matching.

## Data Availability

The training data are derived from publicly available sources: GeoNames (<https://www.geonames.org/>), Wikidata (<https://www.wikidata.org/>), and the Getty Thesaurus of Geographic Names (<https://www.getty.edu/research/tools/vocabularies/tgn/>). The MEHDIE benchmark used for evaluation is available via the supplementary file link at <https://link.springer.com/article/10.1007/s10579-025-09812-9>. Extracted training datasets and pre-computed embeddings will be made available upon publication.

## Code Availability

Training code and inference utilities are available at:

<https://github.com/WorldHistoricalGazetteer/indexing>.

The repository includes technical documentation of the HDBSCAN clustering methodology used for positive pair generation. The system integrates with Elasticsearch for scalable deployment.

## Acknowledgments

This research was supported in part by the University of Pittsburgh Center for Research Computing and Data, RRID:SCR\_022735, through the resources provided. Specifically, this work used the HTC cluster, which is supported by NIH award number S10OD028483, and the H2P cluster, which is supported by NSF award number OAC-2117681.

## References

- Mikel Artetxe, Gorka Labaka, and Eneko Agirre. A robust self-learning method for fully unsupervised cross-lingual mappings of word embeddings. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 789–798, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1073. URL <https://aclanthology.org/P18-1073/>.
- Fernando Benites, Gilbert François Duivesteijn, Pius von Däniken, and Mark Cieliebak. TRANSLIT: A large-scale name transliteration resource. In *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC)*, pages 3265–3271, 2020. URL <https://aclanthology.org/2020.lrec-1.399/>.
- Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a “siamese” time delay neural network. In *Advances in Neural Information Processing Systems (NIPS)*, volume 6, pages 737–744, 1993. URL <https://proceedings.neurips.cc/paper/1993/file/288cc0ff022877bd3df94bc9360b9c5d-Paper.pdf>.
- Sumit Chopra, Raia Hadsell, and Yann LeCun. Learning a similarity metric discriminatively, with application to face verification. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 539–546, 2005. doi: 10.1109/CVPR.2005.202.
- Alexis Conneau, Guillaume Lample, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Word translation without parallel data. In *Proceedings of the International Conference on Learning Representations*, 2018.

- Fangxiaoyu Feng, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. Language-agnostic bert sentence embedding. *arXiv preprint arXiv:2007.01852*, 2022. URL <https://arxiv.org/abs/2007.01852>.
- Karl E. Grossner and Ruth Mostern. Linked places in World Historical Gazetteer. In *Proceedings of the 4th ACM SIGSPATIAL International Workshop on Geospatial Humanities (GeoHumanities)*, pages 40–43, 2021. doi: 10.1145/3486187.3490203.
- Yakov Kolani, Maxim Melichov, Cobi Calev, and Morris Alper. Phonikud: Hebrew grapheme-to-phoneme conversion for real-time text-to-speech. 2025. URL <https://arxiv.org/abs/2506.12311>.
- Grzegorz Kondrak. A new algorithm for the alignment of phonetic sequences. In *Proceedings of the 1st North American Chapter of the Association for Computational Linguistics Conference, NAACL 2000*, page 288–295, USA, 2000. Association for Computational Linguistics.
- Zhouhan Lin, Minwei Feng, Cicero Nogueira Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. URL <https://arxiv.org/abs/1703.03130>.
- David R. Mortensen, Patrick Littell, Akash Bharadwaj, Kartik Goyal, Chris Dyer, and Lori Levin. Panphon: A resource for mapping IPA segments to articulatory feature vectors. In *Proceedings of the 26th International Conference on Computational Linguistics (COLING)*, pages 3475–3484, 2016. URL <https://aclanthology.org/C16-1328/>.
- David R. Mortensen, Siddharth Dalmia, and Patrick Littell. Epitran: Precision G2P for many languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, pages 2634–2639, 2018. URL <https://aclanthology.org/L18-1140/>.
- Jonas Mueller and Aditya Thyagarajan. Siamese recurrent architectures for learning sentence similarity. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2786–2792, 2016. URL <https://dl.acm.org/doi/10.5555/3016100.3016291>.
- Paul Neculoiu, Maarten Versteegh, and Mihai Rotaru. Learning text similarity with siamese recurrent networks. In *Proceedings of the 1st Workshop on Representation Learning for NLP*, pages 148–157, 2016. URL <https://aclanthology.org/W16-1617/>.
- Josef R. Novak, Nobuaki Minematsu, and Keikichi Hirose. Phonetisaurus: Exploring grapheme-to-phoneme conversion with joint n-gram models in the WFST framework. *Natural Language Engineering*, 22(6):907–938, 2016. doi: 10.1017/S1351324915000315.



- Lawrence Philips. Hanging on the metaphone. *Computer Language*, 7(12):39–43, 1990.
- Lawrence Philips. The double metaphone search algorithm. *C/C++ Users Journal*, 18(6):38–43, 2000. doi: 10.5555/349124.349132.
- Qinjun Qiu, Haiyan Li, Xinxin Hu, Miao Tian, Kai Ma, and Yunqiang Zhu. A deep neural network model for chinese toponym matching with geographic pre-training model. *International Journal of Digital Earth*, 17(1):2353111, 2024. doi: 10.1080/17538947.2024.2353111. Proposal of the GeoBERT model.
- Taraka Rama. Siamese convolutional networks for cognate identification. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 1018–1027, 2016. URL <https://aclanthology.org/C16-1097/>.
- Gabriel Recchia and Max Louwerse. A comparison of string similarity measures for toponym matching. In *Vagueness in Communication*, pages 54–67. Springer, 2013. doi: 10.1007/978-3-642-18446-8\_4.
- Robert C. Russell. Index. U.S. Patent 1,261,167, 1918. URL <https://patents.google.com/patent/US1261167A/>. Patented Apr. 2, 1918.
- Tomer Sagi, Moran Zaga, Sinai Rusinek, Marcell Richard Fekete, Johannes Bjerva, and Katja Hose. Utilizing phonetic similarity for cross-source and cross-language toponym matching: a benchmark and prototype. *Language Resources and Evaluation*, 59(3):2427–2451, 2025. doi: 10.1007/s10579-025-09812-9.
- Rodrigo Santos, Patricia Murrieta-Flores, Pablo Calafiore, and Bruno Martins. Learning to combine multiple string similarity metrics for effective toponym matching. *International Journal of Digital Earth*, 11(9):949–965, 2018. doi: 10.1080/17538947.2017.1371253.
- Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823, 2015. doi: 10.1109/CVPR.2015.7298682.
- Jian Zhu, Cong Zhang, and David Jurgens. Byt5 model for massively multilingual grapheme-to-phoneme conversion. 2022. doi: 10.48550/ARXIV.2204.03067. URL <https://arxiv.org/abs/2204.03067>.
- Vilém Zouhar, Kalvin Chang, Chenxuan Cui, Nathaniel Carlson, Nathaniel R. Robinson, Mrinmaya Sachan, and David Mortensen. PWESUITE: Phonetic word embeddings and tasks they facilitate. In *Proceedings of the 2024 Conference on Language Resources and Evaluation (LREC)*, 2024. arXiv:2304.02541.